

IN-VEHICLE INFOTAINMENT (IVI) SYSTEM

Conceptual Architecture and Browser-Based Interactive Simulation

Module 6 · Project 1

Project Report

Submitted by

Annwasha Das

Enrollment No.: 12023002001366

Department of Computer Science and Engineering

Project Repository: <https://github.com/Annwasha2024/ivi-simulation-system>

September 2026

2. Abstract

An In-Vehicle Infotainment (IVI) system combines media playback, navigation, phone connectivity and vehicle information into a single interface for the driver. Its software must coordinate several independent services, share hardware such as speakers and displays, and stay connected to the vehicle network. This project presents a conceptual architecture of an IVI system that integrates media, navigation and projection features, with each component interface and data flow annotated. To make the architecture easier to understand and demonstrate, the static diagram is extended into an interactive browser-based simulation built with only HTML, CSS, vanilla JavaScript and inline SVG.

The simulation models fifteen components, including the driver, HMI, smartphone, central controller, three services, two hardware abstraction layers, GPS, map database, output devices and the vehicle network. Five scenarios can be triggered by the user (Play Music, Start Navigation, Project Route, Incoming Call and End Call), plus a reset. Each scenario highlights the components involved, animates a moving dot along the data path, updates a live status panel and writes timestamped entries to an event log. A small state machine enforces realistic rules, such as rejecting route projection when no route exists and pausing then resuming music around a phone call. Eleven test cases were run against the actual code and all produced the expected results. The project is educational: it does not use a real CAN bus, GPS hardware or Android Automotive OS.

Keywords: In-Vehicle Infotainment, IVI architecture, HMI, middleware, hardware abstraction layer, CAN bus, simulation, SVG.

3. Introduction

3.1 What an IVI System Is and Why It Matters

An In-Vehicle Infotainment system is the group of hardware and software in a car that delivers information and entertainment to the occupants. It usually appears as a central touchscreen, sometimes with a head-up display (HUD), voice control and steering-wheel buttons. Typical functions are audio and radio playback, turn-by-turn navigation, hands-free calling, smartphone projection and display of some vehicle status information.

The IVI system matters for three main reasons:

- **Driver experience:** the IVI is the part of the car that people use the most. A responsive, well organised interface strongly shapes how modern the vehicle feels and is now an important buying factor.

- **Safety:** a poorly designed system distracts the driver. Good IVI design keeps interactions short, gives clear feedback, and lets safety-relevant events such as an incoming call take priority over entertainment.
- **Connectivity:** the IVI links the driver's smartphone, satellite positioning, map data and the vehicle's own electronic units, so it acts as a gateway between the car and the outside world.

3.2 Objective of the Chosen Option

The chosen option (Project 1) asks for a conceptual architecture diagram of an IVI system that integrates media, navigation and projection features, with component interfaces and data flow annotated. The objectives of this work are:

1. To design a layered conceptual architecture showing the driver, HMI, controller/middleware, services, hardware abstraction layers, outputs and vehicle network.
2. To annotate every interface between components with the data that travels over it.
3. To go beyond a static diagram by building an interactive simulation in which the data flow can be watched step by step.
4. To demonstrate cross-feature behaviour, for example a phone call interrupting music, inside the same architecture.

3.3 Scope and Limitations

The work is a conceptual, educational simulation. It covers the architecture, the interactions between components and a small set of demonstration scenarios. It does not attempt to reproduce a production infotainment stack. In particular:

- There is no real vehicle bus; the Vehicle Network (CAN/LIN) and ECUs are shown for architectural completeness only.
- GPS data is simulated and no real GNSS hardware is used.
- There is no Android Automotive OS, and the Audio HAL and Display HAL are represented conceptually rather than implemented.
- The smartphone is a labelled external device; no real Android Auto or Apple CarPlay connection is made.
- All timestamps and log entries are simulated for demonstration.

4. Background / Literature

4.1 Evolution and Role of IVI Systems

Early car audio systems were standalone radios with mechanical controls. Over time they gained CD and digital media support, then integrated navigation, Bluetooth calling and

touchscreens. Today's IVI units are software platforms, often running a full operating system, with multiple applications sharing the screen and speakers. Because these features are handled by different software modules, a coordinating layer (middleware) is normally needed to manage requests, priorities and shared resources. This is the role played by the IVI Controller in the architecture presented here.

4.2 Related Technologies and Standards

- **CAN and LIN buses:** the Controller Area Network (CAN), standardised as ISO 11898, is the main in-vehicle network connecting electronic control units. LIN is a lower-cost bus for simple devices. The IVI reads vehicle state, such as speed or gear, through this network, which is why the Vehicle Network appears in the architecture.
- **Hardware Abstraction Layers (HAL):** a HAL hides hardware details behind a stable software interface so upper layers do not depend on a particular audio chip or display panel. The Audio HAL and Display HAL in this design follow that idea.
- **Android Automotive OS:** a full Android-based operating system that runs on the car's own hardware and includes vehicle-specific APIs, media and navigation frameworks. It is distinct from Android Auto.
- **Android Auto and Apple CarPlay:** projection technologies in which apps run on the phone and their interface is shown on the car display. The Smartphone node and the Projection Service in this project represent this feature.
- **AUTOSAR and COVESA:** AUTOSAR defines standard software architectures for automotive ECUs, while COVESA (formerly GENIVI) is an alliance that promotes open-source infotainment software. Both motivate the layered, service-oriented structure used here.

4.3 Driver Distraction and Interaction Priority

Regulatory and industry guidance on driver distraction, such as the NHTSA visual-manual guidelines for in-vehicle electronic devices, recommends limiting the time and attention that secondary tasks require. This motivates two design choices in the simulation: a simple one-click interaction model, and event priority, in which an incoming call automatically pauses media so the driver is not handling two audio sources at once.

5. Tools & Environment

The simulation deliberately uses only standard web technologies so that it runs offline on any computer without installation.

Item	Details
Languages	HTML5, CSS3, JavaScript (ES2017+, vanilla, no frameworks or libraries)
Diagram	Inline SVG, with paths for interfaces and groups for components

Item	Details
Animation	requestAnimationFrame, SVG getPointAtLength() and CSS transitions
Architecture diagram	Microsoft PowerPoint / Canva (conceptual architecture figure)
Browser	Google Chrome (recommended), Microsoft Edge or Firefox
Network	Not required; the page runs directly from the file system (file://)
Testing	Automated run of the real script.js in a headless DOM (jsdom, Node.js) and a Chromium screenshot (Playwright)
Deliverables	index.html, style.css, script.js, README.md and this report
Source repository	https://github.com/Annwesha2024/ivi-simulation-system

6. System Design / Methodology

6.1 Design Approach

The design follows a layered, hub-and-spoke approach. The driver interacts only with the HMI. The HMI passes every request to a single IVI Controller, which dispatches it to the correct service. Services never talk directly to the driver or to hardware: they deliver their output through a hardware abstraction layer to a physical output device. This keeps responsibilities separate and makes each interface easy to annotate.

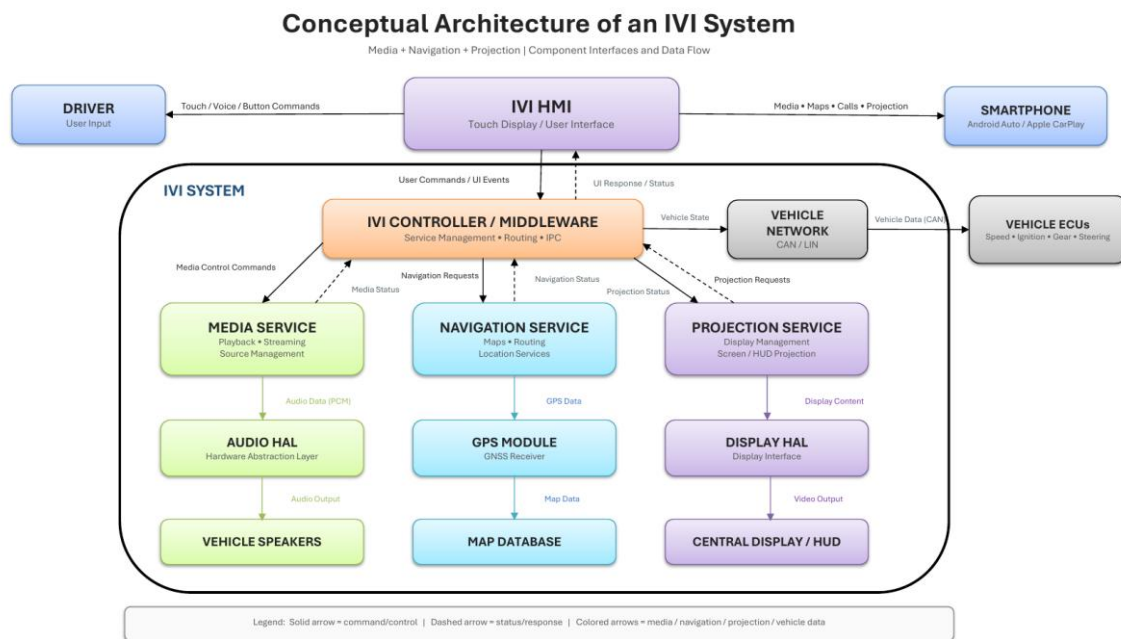


Figure 1: Conceptual architecture of an IVI system (media + navigation + projection)

In Figure 1, solid arrows represent commands and control, dashed arrows represent status or responses, and coloured arrows represent media, navigation, projection or vehicle data. Media components are green, navigation blue, projection purple, external devices light blue and vehicle components grey.

6.2 Components

Component	Purpose
Driver	Provides touch, voice or button input
IVI HMI / Touch Display	User interface the driver sees and touches
Smartphone (Android Auto / CarPlay)	External device sharing media, maps, calls and projection
IVI Controller / Middleware	Central router: service management, routing and inter-process communication
Media Service	Playback, streaming and source management
Navigation Service	Maps, routing and location services
Projection Service	Display management for screen and HUD projection
Audio HAL	Converts media data (PCM) into physical audio
GPS Module	GNSS receiver supplying live coordinates to Navigation
Display HAL	Converts projection data into physical video
Vehicle Speakers	Final audio output
Map Database	Stored map data used for routing
Central Display / HUD	Final visual output
Vehicle Network (CAN/LIN)	Bus between the Controller and vehicle ECUs
Vehicle ECUs	Provide vehicle data such as speed, ignition, gear and steering

6.3 Interfaces and Data Flow

Interface (From → To)	Data carried	Type
Driver → IVI HMI	Touch / Voice / Button commands	Control
IVI HMI ↔ IVI Controller	User commands; UI response / status	Control / Status
Smartphone ↔ IVI HMI	Projection / media data	Data
IVI Controller → Media Service	Media control commands	Control
IVI Controller → Navigation Service	Navigation request	Control
IVI Controller → Projection Service	Projection request	Control
Navigation → Projection Service	Route / map data	Data
Media Service → Audio HAL	Audio data (PCM)	Data
Audio HAL → Vehicle Speakers	Audio output	Data
GPS Module → Navigation Service	GPS data	Data
Map Database → Navigation Service	Map data	Data
Projection Service → Display HAL	Display content	Data
Display HAL → Central Display / HUD	Video output	Data
IVI Controller ↔ Vehicle Network	Vehicle data (CAN)	Data
Vehicle Network ↔ Vehicle ECUs	CAN/LIN messages	Data

GPS Module and Map Database are treated as two independent sources that both feed the Navigation Service. The Map Database does not receive GPS data, and GPS data does not pass through it.

6.4 Behavioural Model

The simulation keeps a small state record: whether music is playing, whether music was paused by a call, whether navigation is active, whether projection is active, whether a call is active, and whether an animation is running. Scenario rules follow from this state:

- Project Route needs an active route; otherwise it is rejected with a message.
- An Incoming Call pauses music only if music was playing, and remembers that it did so.
- End Call resumes music only if the call had paused it; otherwise the system returns to its previous mode.
- While an animation runs, all buttons are disabled so that scenarios cannot overlap.
- Reset returns every component, flag and status field to the idle state.

7. Implementation / Modelling

7.1 Project Structure

File	Role
index.html	Page structure: top bar, status panel, SVG diagram, controls, event log, explanation and project information
style.css	Dark engineering-dashboard theme, layout grid, colour categories and highlight/animation states
script.js	All simulation logic: state, scenarios, animation helpers, status updates and event log
README.md	Overview, run instructions, demo script, limitations and future scope

7.2 Modelling the Architecture in SVG

The whole diagram is written as inline SVG in a 1320×700 coordinate space. Each component is a group (g element) with an id such as n-controller and a category class (cat-media, cat-nav, cat-proj, cat-hub, cat-external or cat-vehicle) that sets its border colour. Each interface is a path element with an id such as e-controller-media. Because the diagram is vector-based it scales with the window, and because nodes and edges have ids, JavaScript can change their appearance simply by adding or removing CSS classes: active (amber, glowing), done (green glow) and paused (red).

7.3 Simulation Logic

script.js is written as a single immediately-invoked function in strict mode. Its main parts are:

- **State object:** stores mode, musicPlaying, musicPausedByCall, navActive, projActive, callActive and busy.
- **animateDot():** uses getTotalLength() and getPointAtLength() on the edge path and requestAnimationFrame to move a glowing circle along the interface, so data flow is visible.
- **playChain():** a generic function that takes a list of steps (node, incoming edge, log message) and plays them in order: mark the edge active, animate the dot, mark the edge done, highlight the node and write a log entry. Most scenarios are described as such a chain.
- **setBusy():** locks all buttons while an animation runs.
- **log() and tickClock():** append a timestamped line to the event log using a simulated clock that starts at 11:31:00 and advances by one or two seconds per entry.
- **setStatus() and setBanner():** update the System Status panel and the coloured scenario banner.

7.4 Scenarios

Scenario	Data path and behaviour
Play Music	Driver → HMI → Controller → Media Service → Audio HAL → Vehicle Speakers. Sets media to PLAYING and mode to MUSIC.
Start Navigation	Driver → HMI → Controller → Navigation Service; then GPS Module → Navigation and Map Database → Navigation; route calculated. Sets navigation to ACTIVE.
Project Route	Navigation → Projection Service → Display HAL → Central Display / HUD. Rejected if no route is active.
Incoming Call	Smartphone → HMI → Controller → Media Service. If music is playing, Media Service, Audio HAL and Speakers turn red (paused).
End Call	Controller notifies Media Service; if music was paused by the call, the session is restored through Audio HAL and Speakers and MUSIC RESUMED is logged.
Reset	Clears all node and edge highlights, the moving dot, every state flag and the status panel.

7.5 User Interface

The screen is divided into a System Status panel (left), the architecture diagram (centre), the Simulation Controls (right), and below them the Event Log, a short explanation of how an IVI works and a project information table. On narrow screens the panels stack in a single column. Buttons have visible keyboard focus, and animation transitions are disabled for users who prefer reduced motion.

8. Testing & Results

8.1 Test Method

The real script.js and index.html were loaded into a headless DOM environment (jsdom on Node.js). Each button was clicked in a planned sequence, and after each scenario finished the System Status panel and banner were read back and compared with the expected values. Animation timers were shortened to speed up the run. A separate run in Chromium (Playwright) was used to capture a screenshot of the working interface.

8.2 Test Cases and Results

No.	Test case	Expected result	Observed result	Result
1	Click Project Route with no route active	Rejected; no projection	Banner: NO ROUTE AVAILABLE — START NAVIGATION FIRST; Projection stays INACTIVE	Pass
2	Play Music from idle	Media PLAYING, mode MUSIC	Mode MUSIC, Media PLAYING, banner MUSIC PLAYING	Pass
3	Start Navigation while music plays	Navigation ACTIVE, mode shows both	Mode MUSIC + NAVIGATION, Navigation ACTIVE	Pass
4	Project Route after navigation	Projection ACTIVE	Projection ACTIVE, banner ROUTE PROJECTED	Pass
5	Incoming Call while music plays	Call ACTIVE, music paused	Mode CALL, Media PAUSED, Call ACTIVE, banner CALL ACTIVE — MUSIC PAUSED	Pass
6	End Call after that call	Music resumes, previous mode restored	Media PLAYING, Call NO CALL, mode MUSIC + NAVIGATION, banner MUSIC RESUMED	Pass
7	End Call when no call exists	No change; message logged	Status unchanged; “No active call to end” added to log	Pass
8	Reset Simulation	Everything returns to idle	Mode IDLE, Media STOPPED, Navigation and Projection INACTIVE, Call NO CALL	Pass
9	Incoming Call with no music	Call ACTIVE, nothing paused	Mode CALL, Media STOPPED, banner CALL ACTIVE	Pass
10	End Call with no music	Returns to IDLE	Mode IDLE, Media STOPPED, banner CALL ENDED	Pass
11	Click another button during an animation	Buttons locked, then released	Buttons disabled during the run and enabled again afterwards	Pass

8.3 Screenshot

Figure 2 shows the simulator after running Play Music, Start Navigation, Project Route and Incoming Call in sequence. Media Service, Audio HAL and Vehicle Speakers are red (paused), the status panel shows CALL / PAUSED / ACTIVE, and the event log records the call sequence.



Figure 2: Simulator state during an active call with music paused

8.4 Results Summary

All eleven test cases passed. The simulation correctly enforces its ordering rule (no projection without a route), handles interaction between features (call interrupting and then restoring media), keeps the status panel consistent with the diagram highlighting, and prevents overlapping animations. These are functional checks of the simulation logic only; no performance or real-vehicle testing was carried out because none is applicable to a conceptual model.

9. Discussion

The simulation shows why a central controller is useful. Every scenario passes through the IVI Controller, so adding a new feature requires only a new service and a new route, not changes to the driver-facing layers. The call-interrupts-music scenario shows the value of this: Media Service does not need to know about the phone; the Controller simply tells it to pause and later resume.

Animating the data flow proved more useful than a static picture for explaining the system. A viewer can see that audio goes through the Audio HAL before reaching the speakers, and that route data reaches the display only through the Projection Service and Display HAL. The event log and status panel reinforce this by giving a textual record of the same sequence.

One deliberate refinement was made when moving from the static diagram to the simulation. In the static diagram, Navigation Service, GPS Module and Map Database are stacked vertically, which could be read as GPS data passing through to the map database. In the

simulation, GPS Module and Map Database are drawn as two separate inputs into the Navigation Service, which better reflects how a navigation service combines a live position with stored map data to plan a route. The dashed status and response arrows from the static diagram are represented in the simulation by the status panel and event log instead of separate return arrows.

The main limitations are those stated earlier: all data is simulated, there is no real bus or hardware, and only five scenarios are modelled. Further, the state model is simple. For example, it does not model call priority levels, multiple audio sources or navigation prompts mixing with music.

10. Conclusion & Future Scope

10.1 Conclusion

This project delivered a conceptual architecture for an In-Vehicle Infotainment system integrating media, navigation and projection, with every component and interface annotated, and turned it into an interactive browser-based simulation. The simulation runs offline using only HTML, CSS, JavaScript and SVG, demonstrates five scenarios with a live status panel and event log, and passed all eleven functional tests. It meets the objectives of explaining the architecture, showing the data flow visually and demonstrating cross-feature behaviour.

10.2 Future Scope

- Integration with real Android Automotive OS components.
- A real CAN bus interface, for example through an OBD-II adapter.
- A Raspberry Pi hardware prototype with a physical touchscreen.
- A real GPS module supplying live coordinates.
- Voice-control input alongside touch.
- Actual Android Auto and Apple CarPlay integration.
- Real vehicle sensor data in place of simulated ECU values.
- Hardware-in-the-loop (HIL) testing with real vehicle components.

11. References

- [1] Bosch, “CAN Specification, Version 2.0,” Robert Bosch GmbH, Stuttgart, 1991.
- [2] ISO 11898-1, “Road vehicles — Controller area network (CAN) — Part 1: Data link layer and physical signalling,” International Organization for Standardization.
- [3] Android Open Source Project, “Android Automotive OS” documentation, source.android.com/docs/automotive.

- [4] Android Developers, “Android for Cars” (Android Auto) documentation, developer.android.com/training/cars.
- [5] Apple Inc., “CarPlay,” apple.com/ios/carplay.
- [6] AUTOSAR, “AUTomotive Open System ARchitecture,” autosar.org.
- [7] COVESA (Connected Vehicle Systems Alliance, formerly GENIVI), covesa.global.
- [8] National Highway Traffic Safety Administration, “Visual-Manual NHTSA Driver Distraction Guidelines for In-Vehicle Electronic Devices,” 2013.
- [9] A. Das, “IVI Simulation System,” GitHub repository, <https://github.com/Annweshha2024/ivi-simulation-system>.
- [10] MDN Web Docs, “SVG,” “SVGGeometryElement.getPointAtLength()” and “window.requestAnimationFrame(),” developer.mozilla.org.

12. Appendix

Appendix A: How to Run the Simulation

1. Download or clone the project from GitHub, or copy the IVI_Simulation folder anywhere on the computer.
2. Repository: <https://github.com/Annweshha2024/ivi-simulation-system>
3. Open the folder and double-click index.html (or right-click and choose Open with Google Chrome).
4. Use the buttons under Simulation Controls to run each scenario. No server or internet connection is needed.
5. For a complete demonstration, run: Play Music, Start Navigation, Project Route, Incoming Call, End Call, then Reset.

Appendix B: Event Log Messages per Scenario

Scenario	Messages written to the log (in order)
Play Music	Driver interaction detected; HMI received media command; Controller routed command to Media Service; Media Service generated audio stream; Audio HAL processing PCM audio; Audio output sent to vehicle speakers; MUSIC PLAYING
Start Navigation	Driver requested navigation; HMI forwarded navigation request; Controller activated Navigation Service; Navigation Service processing request; GPS Module acquired location fix; GPS data received by Navigation Service; Map Database located relevant map tiles; Map data received by Navigation Service; Route calculated; Navigation status updated
Project Route	Navigation route available; Projection Service received route data; Display content prepared; Route projected to Central Display / HUD; ROUTE PROJECTED
Incoming Call	Incoming call detected; Smartphone connection received; IVI Controller received call event; Media Service interrupted — playback paused; Current media playback paused; Call interface activated; CALL ACTIVE — MUSIC PAUSED
End Call	Call ended; Call state cleared; Controller notified Media Service; Previous media session restored; Music playback resumed; MUSIC RESUMED

Appendix C: Key Code Excerpt (script.js)

The generic chain player that drives most scenarios:

```
async function playChain(chain, { holdMs = 260, edgeMs = 420 } = {}) {
  for (let i = 0; i < chain.length; i++) {
    const step = chain[i];
    if (step.edgeFromPrev) {
      markEdge(step.edgeFromPrev, "active");
      await animateDot(step.edgeFromPrev, edgeMs);
      markEdge(step.edgeFromPrev, "done");
    }
    markNode(step.node, "active");
    if (step.msg) log(step.msg, { gapSec: 1 + Math.round(Math.random()) });
    await sleep(holdMs);
  }
}
```

Example of a scenario defined as a chain (Play Music):

```
await playChain([
  { node: "driver", msg: "Driver interaction detected" },
  { node: "hmi", edgeFromPrev: "e-driver-hmi", msg: "HMI received media command" },
  { node: "controller", edgeFromPrev: "e-hmi-controller", msg: "..." },
  { node: "media", edgeFromPrev: "e-controller-media", msg: "..." },
  { node: "audiohal", edgeFromPrev: "e-media-audiohal", msg: "..." },
  { node: "speakers", edgeFromPrev: "e-audiohal-speakers", msg: "..." },
]);
```

The complete source code (index.html, style.css and script.js) is submitted along with this report and is available on GitHub: <https://github.com/Annwesha2024/ivi-simulation-system>